

第9章 图论算法

建议学时:5-6 小时 | 难度:★★★★☆ | 读者背景:已掌握 C 语言,DS&A 零基础

🎯 本章学习目标

- 掌握图的基本术语,说出**邻接矩阵**与**邻接表**的代价差异;完成从 C 的二维数组、手写链表到 Python 嵌套列表、元组的迁移
- 实现**拓扑排序**(入度表 + deque 队列)并用"输出个数 < 顶点数"检测环;实现 BFS 求无权**最短路径**并回溯打印
- 实现 Dijkstra 朴素版与 heap 优化版,说清 INF 选择;实现 Bellman-Ford 检测负环
- 实现 Prim 与 Kruskal **最小生成树**;完成迷宫最短路径综合实验

💡 从 C 到 Python:本章主题如何迁移

图的直觉一句话: **图 = 一堆顶点 + 顶点之间的边**,算法全部建立在"存图、遍历图"两个动作上。前面 8 章学的栈、队列、堆、并查集本章全部派上用场:BFS 与拓扑用第3章的 deque,Dijkstra 用第6章的 heapq,Kruskal 用第8章的不相交——图论是前面所有数据结构的"总演习"。

Python 的解法是"列表 + 元组": `g = [[] for _ in range(n)]` 一行声明动态邻接表;加权图用 `g[u].append((v, w))` 存(终点, 权)元组,顶点不连续时换字典 `g[u].append((v, w))` 同样成立。省去 C 的 malloc/指针,代码接近伪代码,精力全花在算法本身。

📦 前置知识自查

数学前置:无特殊;只需第2章大 O 记号,复杂度字母约定 V = 顶点数、 E = 边数。

C 语言前置:二维数组与下标运算;对照用的 `struct` 链表与 `qsort` 函数指针(可选)。

依赖章节:第3章栈与队列(deque、迭代 DFS 的显式栈);第6章 `heapq` (Dijkstra/Prim 堆优化);第8章不相交集(列表 + 路径压缩)。

🚀 本章学习路线

1. **第一阶段(1.5 小时):**读 §9.1–§9.3,敲两种建图,实现拓扑排序与 BFS
2. **第二阶段(2 小时):**读 §9.4–§9.6,对照朴素/heap 两份 Dijkstra,体会 Prim 的一行之差
3. **第三阶段(1.5 小时):**完成章末实验:迷宫最短路径(BFS + Dijkstra)
4. **验收:**默写建图、BFS 路径回溯、heap 优化 Dijkstra、Kruskal 与各自复杂度

本章知识导图

- §9.1 图的表示:术语与两种存储结构
- §9.2 拓扑排序:入度表与队列
- §9.3 无权最短路径:BFS 与路径回溯
- §9.4 Dijkstra 最短路:贪心、INF 与堆优化
- §9.5 负权边与 Bellman-Ford
- §9.6 最小生成树:Prim 与 Kruskal
- §9.7 最大流与 DFS 应用简介
- §9.8 工程实践:大图的正确打开方式

§9.1 图的表示:术语与两种存储结构

9.1.1 基本术语

图由两部分组成:**顶点(vertex)** 与 **边(edge)**——顶点是"对象",边是"对象之间的关系"。三个常用例子:**城市-航班图**(城市=顶点,航线=边)、**课程依赖图**(课程=顶点,先修=边)、**网页链接图**(网页=顶点,链接=边)。

- **有向图 vs 无向图**:边有方向叫有向边($u \rightarrow v$),全有向的图叫有向图(课程依赖);边不分方向叫无向边($u - v$),对应无向图(道路网)。无向边可看作两条反向有向边。
- **权(weight)**:边上的数值(票价、距离)。带权叫加权图;不带权默认每条边代价 1。
- **路径与环**:顶点间的边序列叫路径;首尾相同的叫环。拓扑排序只能处理无环有向图(DAG),即 §9.2 的主题。

9.1.2 邻接矩阵 vs 邻接表

存图只有两种主流方案,各有胜负。设 V 个顶点、 E 条边, $\deg(u)$ 表示顶点 u 的邻居个数:

对比项	邻接矩阵	邻接表
存储空间	$O(V^2)$,无论边多少都占满	$O(V + E)$,稀疏图省空间
判断边 (u, v) 是否存在	$O(1)$,直接查 $g[u][v]$	$O(\deg(u))$,遍历 u 的链表
遍历 u 的所有邻居	$O(V)$,扫一整行	$O(\deg(u))$,顺着链表走
适用场景	稠密小图($V \leq 2000$)	稀疏大图(工程绝大多数)

口诀:图算法都要"遍历邻居",邻接表把这一步从 $O(V)$ 降到 $O(\deg)$,整体复杂度下降一个量级。邻接矩阵的用武之地是" V 小且频繁查边"的场景(如 Floyd)。 $V = 10^5$ 时矩阵要 10^{10} 个格子 ≈ 10 GB,直接内存崩溃。

C 的写法	Python 的写法
<code>int g[100][100];</code> 大小写死的二维数组	<code>g = [[0] * n for _ in range(n)]</code> 嵌套列表
<code>struct Edge { int to, w; struct Edge* next; };</code> 手写链表	<code>g = [[] for _ in range(n)]</code> 加权用 <code>(v, w)</code> 元组
<code>for (Edge* e = head[u]; e; e = e->next)</code> 指针遍历	<code>for v in g[u]</code> 直接遍历
<code>malloc / free</code> 手动管理,忘记即泄漏	垃圾回收自动管理,无泄漏

示例 9-1 有向无权图:邻接表建法(矩阵建法见上表 C 对照)

```
# 邻接表:g[u] 是 u 的所有出边终点
def build_list(n, edges):
    g = [[] for _ in range(n)]      # n 个独立空列表,勿写 [[]]*n
    for u, v in edges:
        g[u].append(v)             # 无向图再加 g[v].append(u)
    return g

# 使用:g = build_list(4, [(0,1), (0,2), (2,3)])
# 遍历 u 的邻居:for v in g[u]
```

9.1.3 加权图:C 的手写链表 → Python 的元组

加权图在 C 里要用"边结点带权"的链表,建图几十行;Python 用 (终点, 权) 元组: `g[u].append((v, w))` ——读边直接 `append`,第6章 Dijkstra 的图就存成这种结构。注意嵌套列表的坑: `[[0] * n] * n` 会让所有行共享同一对象,必须先 `[[] for _ in range(n)]` 逐行新建。

§9.2 拓扑排序:入度表与队列

9.2.1 什么是拓扑排序

拓扑排序:把有向无环图(DAG)的所有顶点排成一列,使每条边 `u → v` 都满足"u 排在 v 前"。典型场景是**课程先修依赖**:C 在数据结构前,数据结构在算法分析前——课程表就是拓扑序。**带环的有向图不存在拓扑序**(A 先修 B、B 又先修 A,谁也排不了),所以"能否拓扑排序"等价于"有没有环"。

9.2.2 Kahn 算法:入度表 + 队列

算法直觉:"不断摘掉没有前置的课程"。三步:① 统计每个顶点的**入度**(指向它的边数);② 入度为 0 者全部入队——无前置,随时可安排;③ 出队 `u`,删掉它的所有出边(邻居入度减 1),新入度为 0 的邻居入队。**无环时必然输出全部 V 个顶点;输出个数 < V 就是有环,务必检查**。

示例 9-2 拓扑排序(Kahn 算法)与环检测

```

from collections import deque

# 返回拓扑序列;有环返回空序列
def topo_sort(n, g):
    indeg = [0] * n          # ① 入度表
    for u in range(n):
        for v in g[u]:
            indeg[v] += 1
    q = deque(u for u in range(n) if indeg[u] == 0)
    order = []
    while q:
        u = q.popleft()
        order.append(u)
        for v in g[u]:      # 删掉 u 的出边
            indeg[v] -= 1
            if indeg[v] == 0: # 前置课程全部完成才入队
                q.append(v)
    if len(order) != n: return [] # 输出个数 < n:有环!
    return order

```

§9.3 无权最短路径:BFS 与路径回溯

9.3.1 为什么 BFS 能求无权最短路

无权图(每条边代价 1)中,**最短路径**就是"边数最少的路径"。BFS 按层扩展:第 1 层距离 1,第 2 层距离 2.....**每个顶点第一次被访问时,步数一定最少**——队列严格按距离递增出队,"第一次到达即最短"正是 BFS 不需要 Dijkstra 的理由。

C 的手写循环队列	Python 的 collections.deque
<pre>int q[100000]; int head, tail;</pre> 自己维护下标与取模	<pre>q = deque([s]); q.popleft()</pre>
空间写死,溢出要自己检查	自动扩容,永不"满"
空/满判断:计数法或浪费一格(第3章)	while q: 一行判空

9.3.2 距离数组与前驱数组回溯

求步数只需 `dist` 数组;打印路径还要 `prev` 数组记录"每个顶点从谁到达"。到终点后沿 `prev` 回溯到起点再反转,就是完整路径——**前驱数组回溯**,Dijkstra 与 Bellman-Ford 沿用同一套路。

示例 9-3 无权最短路:BFS + 前驱数组回溯

```

from collections import deque

# 求 s 到各点最短距离;dist[i] = -1 表示不可达
def bfs_dist(s, g):
    n = len(g)
    dist = [-1] * n          # -1 兼任 visited
    prev = [-1] * n          # prev[v]:从谁到达 v
    dist[s] = 0
    q = deque([s])
    while q:
        u = q.popleft()
        for v in g[u]:
            if dist[v] != -1: continue    # 第一次到达即最短
            dist[v] = dist[u] + 1
            prev[v] = u
            q.append(v)
    return dist, prev

# 打印 s 到 t 的路径(沿 prev 回溯)
def print_path(s, t, prev):
    path = []
    x = t
    while x != -1:
        path.append(x)
        if x == s: break
        x = prev[x]
    if not path or path[-1] != s:    # 回溯到 -1 也没到 s
        print("无路径可达"); return
    print(" -> ".join(map(str, reversed(path))))

```

实现建议:dist 兼任 visited

BFS 里 `dist[v] != -1` 既判"访问过"又存距离,一个数组干两件事,无需单独写 `visited` ——与第3章"循环队列用 `count` 判空满"同一思想:状态能合并就合并。回溯打印注意:路径数组是"终点→起点"顺序,输出前要反转。

§9.4 Dijkstra 最短路:贪心、INF 与堆优化

9.4.1 贪心思想与正确性直觉

加权图(边权非负)的单源最短路由 Dijkstra 解决,思路是 **贪心**:每轮从"还没确定最短距离"的顶点中,挑距离最小的确定为**答案**。为什么敢确定?边权全非负——任何"经过别的未确定顶点"的绕路,起步就比这个距离大,不可能更短。"确定"是永久的:顶点一旦被选出,dist 就是最终答案,只增不改。"每次取当前最小"正是第6章小顶堆的用武之地。

9.4.2 INF 的坑:float('inf') 一劳永逸

dist 初始值必须是一个"表示无穷大"的数。Python 用 `float('inf')`:任何有限数加 `inf` 仍是 `inf`,不存在 C++ 里 `INT_MAX + w` 溢出成负数的问题(C++ 版 §9.4 详述了那个坑,要改用 `1e9` 或 `long long`)。

⚠ 误区 3:把 INF 写成有限大整数

有的同学把 C++ 习惯带过来,写 `INF = 10**18`:边权为负时 `dist[u] + w` 可能小于 `INF`,把"不可达"误判成"可达";更隐蔽的是 `int` 与 `float` 混用,类型与比较行为都变得不可预测。统一用 `float('inf')`:比较、加法全部安全,最后输出前再转 `int`。

C 的写法	Python 的写法
<code>#define INF 0x3f3f3f3f</code> 魔法常量,靠记忆	<code>INF = float('inf')</code> 内置无穷大
<code>memset(dist, 0x3f, sizeof dist)</code> 技巧性初始化	<code>dist = [INF] * n</code> 列表乘法一行填充
手写 <code>for</code> 循环扫描找最小, $O(V)$ 每次	<code>heapq.heappop</code> 堆顶即最小, $O(\log V)$ 每次

9.4.3 朴素版与堆优化版并排:手写实现 vs 标准库

两段代码解决同一问题,骨架完全相同,差异只在"找当前最小"——正是本章的"手写实现 + 标准库写法"并排:朴素版每轮全数组扫描,总代价 $O(V^2)$;堆优化版把"找最小"委托给第6章的 `heapq`,总代价 $O(E \log V)$,大图必须用它。

示例 9-4 Dijkstra 朴素版 $O(V^2)$:手写扫描找最小

```
def dijkstra_naive(s, g):
    n = len(g)
    INF = float('inf')
    dist = [INF] * n
    done = [False] * n          # done[u]:u 已确定
    dist[s] = 0
    for _ in range(n):
        u = -1
        for i in range(n):      # 扫描:未确定者中 dist 最小
            if not done[i] and (u == -1 or dist[i] < dist[u]):
                u = i
        if u == -1 or dist[u] == INF: break  # 其余顶点不可达
        done[u] = True          # 确定 u,永不再改
        for v, w in g[u]:       # 松弛:尝试用 u 更新邻居
            if dist[v] > dist[u] + w:
                dist[v] = dist[u] + w
    return dist
```

示例 9-5 Dijkstra 堆优化版 $O(E \log V)$:标准库 `heapq`

```
import heapq

def dijkstra_heap(s, g):
    n = len(g)
    INF = float('inf')
    dist = [INF] * n
    dist[s] = 0
    pq = [(0, s)]          # (距离, 顶点), 小顶堆
    while pq:
        d, u = heapq.heappop(pq)
        if d > dist[u]: continue    # 懒删除:过期的旧记录直接跳过
        for v, w in g[u]:
            if dist[v] > d + w:
                dist[v] = d + w
                heapq.heappush(pq, (dist[v], v)) # 允许重复入堆
    return dist
```

实现建议:懒删除(惰性删除)是堆优化 Dijkstra 的标配

同一顶点可能被松弛多次、在堆里留下多份记录。堆优化版**不删除旧记录**,出堆时检查 $d > \text{dist}[u]$ 的过期记录直接丢弃——这叫懒删除,正确性由"旧记录距离必然不小于新记录"保证。heapq 没有 decreaseKey 接口,懒删除正是替代。套路:push 不查重,pop 查过期。

9.4.4 复杂度与选型

版本	"找最小"的手段	总复杂度	$V = 10^5$ 、 $E = 10^6$ 时的量级
朴素	每轮全数组扫描 $O(V)$	$O(V^2)$	10^{10} ,不可用
堆优化	堆顶 $O(\log V)$	$O(E \log V)$	约 2×10^7 ,秒级

选型结论:稠密小图($V \leq 2000$)两者皆可,朴素版常数更小; V 上万或 E 稀疏一律堆优化,工程与竞赛的默认写法。

§9.5 负权边与 Bellman-Ford

9.5.1 负权边为何破坏 Dijkstra

三顶点例子:边 $s \rightarrow a$ 权 2、 $s \rightarrow b$ 权 3、 $b \rightarrow a$ 权 -2(有向)。Dijkstra 第一轮挑出 $a(d=2)$ 并永久确定;第二轮处理 b 时松弛 $a: 3+(-2)=1 < 2$,但 a 已"确定"——真实最短路 $s \rightarrow b \rightarrow a$ 只有 1,答案错了。负权边让"绕路"反而更便宜,贪心的"确定即最终"被推翻。结论:Dijkstra 只适用于非负权图。

9.5.2 Bellman-Ford: $V-1$ 轮松弛

Bellman-Ford 的思想朴素而彻底:每轮对全部 E 条边松弛一次,共做 $V-1$ 轮。为什么够?无环路径至多 $V-1$ 条边:第 1 轮确定"1 条边"的最短路,第 k 轮由第 $k-1$ 轮答案再延伸一条边,第 k 轮后"至多 k 条边"的路径都被考虑。复杂度 $O(V \cdot E)$,能处理负权,代价是不能有负环。

示例 9-6 Bellman-Ford: $V-1$ 轮松弛 + 负环检测

```
def bellman_ford(s, edges, n):
    INF = float('inf')
    dist = [INF] * n
    dist[s] = 0
    for _ in range(n - 1):          # 至多 V-1 轮
        changed = False
        for u, v, w in edges:      # 每轮扫全部边
            if dist[u] != INF and dist[v] > dist[u] + w:
                dist[v] = dist[u] + w
                changed = True
        if not changed: break      # 无更新即可提前结束
    for u, v, w in edges:          # 第 V 轮:还能松弛 = 有负环
        if dist[u] != INF and dist[v] > dist[u] + w:
            return None            # 返回 None 表示有负环
    return dist
```

⚠ 误区 4:负环检测忘写,或写进主循环里

负环检测必须在 $V-1$ 轮**结束之后**单独做:负环让距离无限变小,检测混进主循环会在正常图误报"有环"。另一易错点:松弛前跳过 `dist[u] == INF` 的边——inf 减负权仍是 inf,不判断会得到假答案。

§9.6 最小生成树:Prim 与 Kruskal

9.6.1 定义与两种思路

最小生成树(MST):在连通加权无向图中,选 $V-1$ 条边把全部顶点连成一棵树(无环),边权总和最小。应用:铺电缆、修公路,让所有城市连通且总造价最低。两条路线:Prim"加点"——每次并入"离树最近"的顶点;Kruskal"加边"——边按权排序,从小到大挑不成环的边。

9.6.2 Prim:与 Dijkstra 只差一行

把 §9.4 的 Dijkstra 朴素版抄过来,Prim 与它几乎一模一样——唯一区别是松弛标准:

算法	key[u] 的含义	松弛标准
Dijkstra	起点到 u 的当前最短距离	$\text{dist}[v] > \text{dist}[u] + w$ (累加路径)
Prim	u 到"已选集合"的最近距离	$\text{key}[v] > w$ (只看边权,不累加)

Prim 的 key 是"顶点到树集合的距离",更新只比较**这条边本身**的权;Dijkstra 比较"起点到 u 的整条路径 + 边权"。一行之差,两种算法,也是第10章贪心范式的两个变体。

示例 9-7 Prim 朴素版 $O(V^2)$:与 Dijkstra 只差松弛标准


```
def prim_mst(s, g):
    n = len(g)
    INF = float('inf')
    key = [INF] * n          # 到"已选集合"的最短距离
    in_tree = [False] * n
    key[s] = 0
    total = 0
    for _ in range(n):
        u = -1
        for i in range(n):    # 扫描:离树最近的未入树顶点
            if not in_tree[i] and (u == -1 or key[i] < key[u]):
                u = i
        if u == -1 or key[u] == INF: break    # 图不连通
        in_tree[u] = True
        total += key[u]
        for v, w in g[u]:
            if not in_tree[v] and w < key[v]: # 松弛标准:直接比边权!
                key[v] = w
    return total              # 生成树总权
```

9.6.3 Kruskal:边排序 + 不相交集

Kruskal 更"贪":边按权升序,从小到大逐条考察,只要两 endpoints 还没连通就收下——"是否连通"正是第8章不相交集的看家本领(find/union 均摊近 $O(1)$);已连通则收下必成环,必须跳过。总复杂度 $O(E \log E)$,由排序主导。

对照 C 的 `qsort` 函数指针,这里用 `sort` + `key`: `edges.sort(key=lambda e: e[2])` ——"按哪一项排"写成一个 lambda,一行搞定,写错编译期报错。

示例 9-8 Kruskal:排序 + 不相交集(第8章的 find/unite 直接拿来用)

```
def kruskal_mst(n, edges):
    parent = list(range(n))

    def find(x):
        if parent[x] != x:
            parent[x] = find(parent[x])    # 路径压缩
        return parent[x]

    def unite(a, b):
        a, b = find(a), find(b)
        if a == b: return False            # 已连通:这条边会成环
        parent[a] = b
        return True

    edges.sort(key=lambda e: e[2])        # ① 按边权升序
    total = cnt = 0
    for u, v, w in edges:
        if unite(u, v):                    # ② 两端点不连通才收下
            total += w
            cnt += 1
        if cnt == n - 1: break             # 树有 V-1 条边即完成
    return total if cnt == n - 1 else -1   # -1 表示图不连通
```

⚠ 误区 5:把 Prim 的松弛标准抄成 Dijkstra 的

两段代码结构高度相似,最常见的复制粘贴事故是把 `w < key[v]` 抄成 `dist[u] + w < key[v]`。自查口诀: Prim 只比"这一条边",Dijkstra 比"整条路径";写完用三顶点小例子手算验证。

§9.7 最大流与 DFS 应用简介

9.7.1 网络流:一句话

最大流问题:有向加权图中,从源点 s 到汇点 t 能运输的最大流量(每条边有容量上限)。一句话直觉:反复找"还能增流的路径"(增广路),直到找不到为止,累加的流量就是最大流。用 BFS 找最短增广路的版本叫 **Edmonds-Karp**,复杂度 $O(V \cdot E^2)$,Dinic、ISAP 都是它的优化。应用:交通调度、二分图匹配。

9.7.2 DFS 应用:割点与双连通

两个概念:**割点**——无向图中删掉它后图不再连通的顶点(如通信网络的关键路由器);**双连通**——删掉任意一个顶点仍连通的图,其极大子图叫双连通分量。求割点的经典算法(Tarjan)基于 DFS 的"时间戳"思想,教材有完整讲解。记住一句:DFS 访问顶点的顺序本身携带结构信息(发现时间、完成时间),能回答"连通性"类问题。

9.7.3 迭代版 DFS:显式栈

递归 DFS 在深图(如 10^5 层链)上会撑爆系统调用栈;第3章说过"递归都能改写为显式栈",这里实战:显式栈压"待访问的顶点",与函数调用栈压"返回地址"同一思想。图遍历必须带 visited:无向边让 u 、 v 互相"发现",不带就无限往返;迭代版同顶点可能被 push 多次,出栈时去重。

示例 9-9 深度优先遍历:递归版与显式栈迭代版(第3章)

```
def dfs_rec(u, g, vis):
    vis[u] = True                # 访问当前顶点
    print(u, end=' ')
    for v in g[u]:
        if not vis[v]:
            dfs_rec(v, g, vis)   # 递归:深度优先

def dfs_iter(s, g):
    vis = [False] * len(g)
    st = [s]                    # 显式栈代替系统调用栈
    while st:
        u = st.pop()
        if vis[u]: continue     # 可能被压入多次,去重
        vis[u] = True
        print(u, end=' ')
        for v in g[u]:
            if not vis[v]:
                st.append(v)     # 注意:压栈顺序与递归相反
```

§9.8 工程实践:大图的正确打开方式

9.8.1 大图选型:邻接表优先

真实世界的图几乎都稀疏(社交网络、网页链接皆如此),三条准则:

- **存图用邻接表:** $V \geq 10^4$ 一律邻接表,矩阵只在小稠密图出现(§9.1 误区1)
- **无向图建两次边:** $g[u]$ 与 $g[v]$ 互相 append,漏一边就成"单行道"
- **顶点编号连续:** 用 $0..V-1$ 整数做下标;字符串先映射成编号(第5章)

本章复杂度速查表

操作	数据结构 / 算法	平均	最坏
遍历所有顶点与边	DFS / BFS	$O(V + E)$	$O(V + E)$
查边 (u, v)	邻接矩阵	$O(1)$	$O(1)$
查边 (u, v)	邻接表	$O(\deg(u))$	$O(\deg(u))$
存储	邻接矩阵	$O(V^2)$	$O(V^2)$
存储	邻接表	$O(V + E)$	$O(V + E)$
拓扑排序	Kahn(入度表 + 队列)	$O(V + E)$	$O(V + E)$
无权最短路径	BFS	$O(V + E)$	$O(V + E)$
单源最短路径(非负权)	Dijkstra 朴素	$O(V^2)$	$O(V^2)$
单源最短路径(非负权)	Dijkstra 堆优化	$O(E \log V)$	$O(E \log V)$
单源最短路径(可负权)	Bellman-Ford	$O(V \cdot E)$	$O(V \cdot E)$
负环检测	Bellman-Ford 第 V 轮	$O(V \cdot E)$	$O(V \cdot E)$
最小生成树	Prim 朴素	$O(V^2)$	$O(V^2)$
最小生成树	Prim 堆优化	$O(E \log V)$	$O(E \log V)$
最小生成树	Kruskal(排序 + 并查集)	$O(E \log E)$	$O(E \log E)$
最大流	Edmonds-Karp(BFS 增广)	$O(V \cdot E^2)$	$O(V \cdot E^2)$

最小必会清单

- 说出顶点、边、有向/无向、权、路径、环六个术语并配现实例子
- 独立写出邻接矩阵与邻接表两种建图代码,说出空间与查询复杂度;避开 `[[] ] * n` 共享列表的坑
- 默写 Kahn 拓扑排序,解释"输出个数 $< V$ 则有环"的检测原理
- 默写 BFS 与 Dijkstra(朴素/heap 优化),都用前驱数组回溯打印路径,解释贪心正确性的直觉(非负权)
- 解释 `float('inf')` 与 `C++ INT_MAX` 两种 INF 方案的差异
- 说出负权边如何破坏 Dijkstra,默写 Bellman-Ford 的松弛与负环检测
- 说出 Prim 与 Dijkstra 的唯一区别(松弛标准),默写 Kruskal(排序 + 并查集)
- 写出迭代版 DFS(显式栈),解释 `visited` 为何必不可少

自测题

Q 1. $V = 10^5$ 、 $E = 2 \times 10^5$ 的稀疏图,邻接矩阵与邻接表各占多少空间?查一条边各是什么复杂度?

✅ 参考答案

矩阵 $O(V^2) \approx 10^{10}$ 格(数十 GB,不可行),查边 $O(1)$;邻接表 $O(V+E) \approx 3 \times 10^5$,查边 $O(\deg(u))$ 平均 $O(E/V)$ 。稀疏大图必须用邻接表。

Q 2. 拓扑排序的队列换成栈,结果还是合法拓扑序吗?为什么环检测不受影响?

✅ 参考答案

仍然合法:入度为 0 的顶点可按任意顺序安排,只要"先修在前"成立即可。环检测只看输出个数是否等于 V ,与出队顺序无关。

Q 3. Dijkstra 朴素版与堆优化版都正确,复杂度为何差一个量级?"懒删除"怎么工作的?

✅ 参考答案

差别在"找当前最小":朴素每轮扫描 $O(V)$,总 $O(V^2)$;堆优化用 heapq ,堆顶 $O(1)$,总 $O(E \log V)$ 。懒删除:顶点被松弛多次会在堆里留多份记录,出堆时把 $d > \text{dist}[u]$ 的旧记录丢弃——旧记录距离不小于新记录,不影响正确性。

Q 4. 为什么 Bellman-Ford 恰好做 $V-1$ 轮松弛就够?第 V 轮检查到什么说明有负环?

✅ 参考答案

无环路径至多 $V-1$ 条边,第 k 轮后"至多 k 条边"的路径都被考虑过, $V-1$ 轮后全部松弛完毕。若第 V 轮仍有边能松弛,说明存在比" $V-1$ 条边上限"还短的路径——只能是有负环(可无限绕圈变短)。

Q 5. Prim 与 Dijkstra 的代码几乎相同,区别在哪一行?Kruskal 里并查集的 `unite` 返回 `false` 意味着什么?

✅ 参考答案

区别在松弛标准:Prim 的 `key` 是"到已选集合的距离",更新只比较边权(`key[v] > w`);Dijkstra 的 `dist` 是"起点的距离",更新时累加路径(`dist[v] > dist[u] + w`)。Kruskal 中 `unite` 返回 `false` 表示两端点已连通,收下会成环,必须跳过。

章末实验题:迷宫最短路径(BFS + Dijkstra)

📄 实验 9:迷宫最短路径(BFS + Dijkstra)

实验要求:

- 任务 1:读入/内置迷宫并转成图:顶点 = 格子,边 = 四方向可通行相邻格(\$9.1)
- 任务 2:BFS 求 S 到 E 的最短步数路径,前驱数组回溯并打印(\$9.3)
- 任务 3:给地形设通过代价(沼泽 ~ 代价 5),Dijkstra 堆优化求最低代价路径(\$9.4)
- 任务 4:输出两种路径的步数与代价对比,解释为何可能不同(\$9.3 + \$9.4)
- 任务 5:终点不可达时输出"无解"而不崩溃(\$9.8)

实验环境:Python 3。运行: `python lab9.py` (内置)、`python lab9.py sealed` (无解)、`python lab9.py maze.txt` (文件)。

第一步 read_maze 与邻居枚举(越界、墙检查),用 find_pos 求 S、E 坐标

第二步 S 用 deque,prev 字典兼任 visited;到 E 后回溯打印

第三步ijkstra 用 heapq,INF 用 float('inf');懒删除(出堆时 $d > \text{dist}[u]$ 跳过)且不松弛墙

第四步 比行统计两条路径的真实代价,体会"步数最少"与"代价最低"可能是两条路

✔ 参考答案(完整可运行程序,分三段)

```

# lab9.py — 迷宫最短路径(BFS + Dijkstra)
# 运行: python lab9.py [sealed|maze.txt]
import heapq
import sys
from collections import deque

# 内置迷宫:'#' 墙, '.' 平地(代价 1), '~' 沼泽(代价 5), 'S' 起点, 'E' 终点
# 设计: 上走廊短(12 步)但有两格沼泽(代价 21);
#       下走廊长(18 步)却全平地(代价 18)—BFS 选短,Dijkstra 选廉。
MAZE = """\
#####
#S...~..#
#.#.###.#
#.#.###.#
#.#...##
#.#.###.#
#.#.###.#
#...##E##
#####
#####\
"""

SEALED = """\
#####
##S##
#.#.##
#.E.#
#####\
"""

COST = {'S': 1, 'E': 1, '.': 1, '~': 5}

def read_maze(arg):
    if arg == "sealed":
        return SEALED.splitlines()
    if arg:
        with open(arg, encoding="utf-8") as f:
            return [line.rstrip("\n") for line in f if line.strip()]
    return MAZE.splitlines()

def find_pos(maze, ch):
    for r, row in enumerate(maze):
        for c, cell in enumerate(row):
            if cell == ch:
                return r, c
    return -1, -1

def neighbors(maze, r, c):
    """任务 1: 网格 -> 图, 四方向邻居即边(隐式图)。"""
    h, w = len(maze), len(maze[0])
    for dr, dc in ((-1, 0), (1, 0), (0, -1), (0, 1)):
        nr, nc = r + dr, c + dc

```

```
if 0 <= nr < h and 0 <= nc < w and maze[nr][nc] != '#':  
    yield nr, nc
```

| lab9.py(第二段:BFS 与 Dijkstra 两个算法)


```

def bfs_shortest(maze, s, t):
    """任务 2:无权最短路(BFS)。返回 (步数, 路径);无解返回 None。"""
    q = deque([s])
    prev = {s: None}
    while q:
        u = q.popleft()
        if u == t:
            break
        for v in neighbors(maze, *u):
            if v not in prev:          # 第一次到达即最短
                prev[v] = u
                q.append(v)
    if t not in prev:
        return None                  # 任务 5:终点不可达
    path = []
    x = t
    while x is not None:            # 前驱数组回溯
        path.append(x)
        x = prev[x]
    path.reverse()
    return len(path) - 1, path

def dijkstra_cost(maze, s, t):
    """任务 3:最低代价路径(Dijkstra 堆优化)。返回 (总代价, 路径);无解返回 None。"""
    h, w = len(maze), len(maze[0])
    INF = float('inf')              # Python 用无穷大,无溢出问题
    dist = [[INF] * w for _ in range(h)]
    prev = {}
    dist[s[0]][s[1]] = 0
    pq = [(0, s)]                   # (当前代价, 坐标),小顶堆
    while pq:
        d, u = heapq.heappop(pq)
        if d > dist[u[0]][u[1]]:
            continue                # 懒删除:过期记录直接跳过
        if u == t:
            break
        for v in neighbors(maze, *u):
            nd = d + COST[maze[v[0]][v[1]]]
            if nd < dist[v[0]][v[1]]:
                dist[v[0]][v[1]] = nd
                prev[v] = u
                heapq.heappush(pq, (nd, v))
    if dist[t[0]][t[1]] == INF:
        return None
    path = []
    x = t
    while x is not None:
        path.append(x)
        x = prev.get(x)
    path.reverse()
    return dist[t[0]][t[1]], path

```

lab9.py(第三段:主程序,任务 4/5)

```

def draw_path(maze, path):
    """把路径画回迷宫:* 标出经过的格子。"""
    grid = [list(row) for row in maze]
    for r, c in path:
        if grid[r][c] not in 'SE':
            grid[r][c] = '*'
    return [' '.join(row) for row in grid]

def main():
    arg = sys.argv[1] if len(sys.argv) > 1 else ""
    maze = read_maze(arg)
    s, t = find_pos(maze, 'S'), find_pos(maze, 'E')
    if s == (-1, -1) or t == (-1, -1):
        print("迷宫缺少起点 S 或终点 E")
        return
    print(f"迷宫 {len(maze)} 行 x {len(maze[0])} 列,起点 {s},终点 {t}")
    print("图例: # 墙 . 平地(代价1) ~ 沼泽(代价5) S/E 起点/终点")

    res_bfs = bfs_shortest(maze, s, t)
    if res_bfs is None:
        print("BFS : 无解—起点到终点不可达(被墙封死)")
    else:
        steps, path = res_bfs
        print(f"BFS : 最短步数 = {steps}")
        print("  路径:", " -> ".join(f"({r},{c})" for r, c in path))
        for row in draw_path(maze, path):
            print("    ", row)

    res_dij = dijkstra_cost(maze, s, t)
    if res_dij is None:
        print("Dijkstra: 无解")
    else:
        cost, path = res_dij
        print(f"Dijkstra: 最低总代价 = {cost},步数 = {len(path)-1}")
        print("  路径:", " -> ".join(f"({r},{c})" for r, c in path))
        for row in draw_path(maze, path):
            print("    ", row)

    # 任务 4:两种路径对比
    if res_bfs and res_dij:
        steps, path = res_bfs
        cost, path2 = res_dij
        bfs_cost = sum(COST[maze[r][c]] for r, c in path)
        print(f"对比: BFS 最短步数 {steps}(路径代价 {bfs_cost}) vs "
              f"Dijkstra 最低代价路径步数 {len(path2)-1}(代价 {cost})")

if __name__ == "__main__":
    main()

```

示例运行输出(真实运行结果):

```
$ python lab9.py
迷宫 10 行 x 10 列,起点 (1, 1),终点 (7, 7)
BFS : 最短步数 = 12
    路径: (1,1) -> (1,2) -> (1,3) -> (1,4) -> (1,5) -> (1,6) -> (1,7) -> (2,7) -> (3,7) -> (4,7) -> (5,7) -> (6,7) -> (7,7)
Dijkstra: 最低总代价 = 18,步数 = 18
    路径: (1,1) -> (2,1) -> (3,1) -> (4,1) -> (5,1) -> (6,1) -> (7,1) -> (7,2) -> (7,3) -> (7,4) -> (6,4) -> (5,4) -> (4,4) -> (4,5) -> (4,6) -> (4,7) -> (5,7) -> (6,7) -> (7,7)
对比: BFS 最短步数 12(路径代价 21) vs Dijkstra 最低代价路径步数 18(代价 18)

$ python lab9.py sealed
BFS : 无解—起点到终点不可达(被墙封死)
Dijkstra: 无解
```

设计说明:① 迷宫直接以 (行, 列) 元组为顶点,dist 用二维列表,免去编号换算;② "四方向邻居"即边,不显式建邻接表,是"隐式图"代表;③ BFS 与 Dijkstra 共用 prev 回溯,体现 §9.3/§9.4 的算法家族关系;④ INF 用 float('inf'),无溢出问题;⑤ 内置"无解"样例:两算法都检查终点距离,输出提示不崩溃。

下一章预告:第10章 算法设计技巧——贪心、分治、动态规划、回溯四大范式。Dijkstra 与 Prim 的贪心本质将在那里升华。